

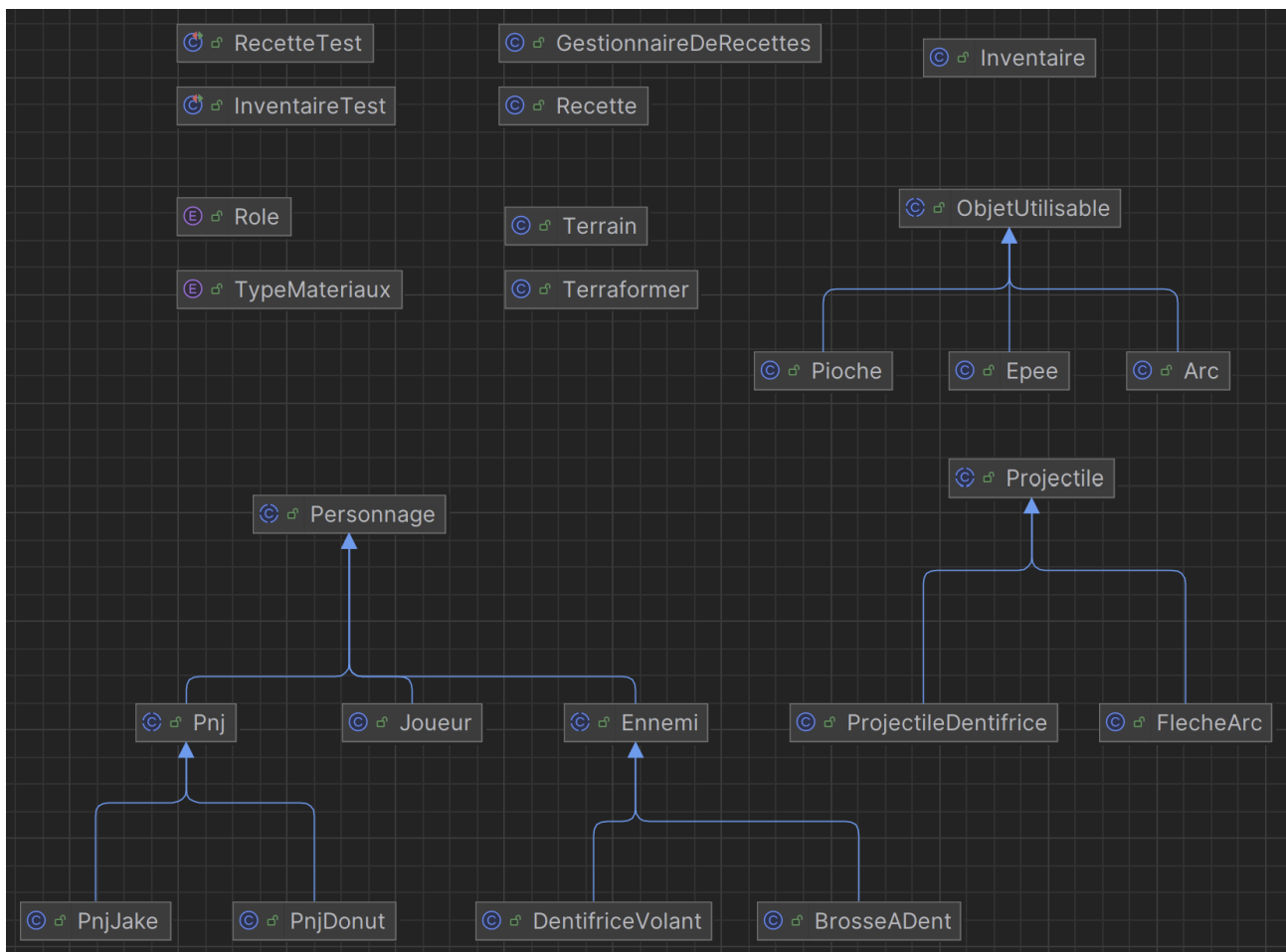
Documents Pour les SAEs S2 2024-2025

1 Auteurs

ABERKANE Adam - DIALLO Djeneba - BEN MOUSSA Wael

2 Diagrammes de classe (15 points pour SAEs S2.01 S2.02)

1 : Vous donnerez une vision d'ensemble de la partie modèle de votre programme à l'aide d'un ou de plusieurs diagrammes de classe commentés.



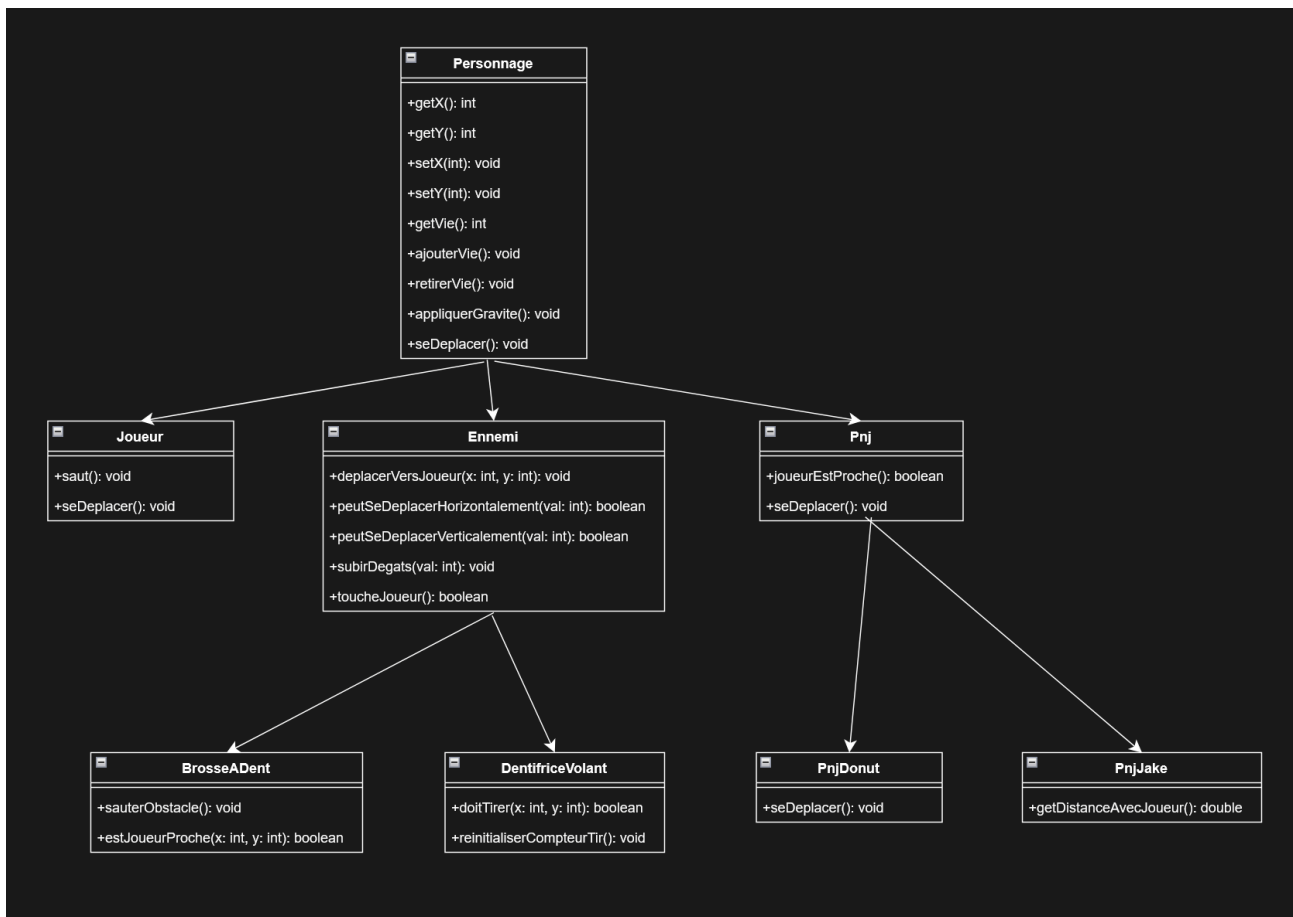
Le diagramme de classe général que nous avons réalisé donne une vue d'ensemble claire de la structure du modèle de notre jeu. Il montre les différentes classes principales utilisées pour faire fonctionner les mécaniques du jeu, ainsi que les relations entre elles. Au centre, on retrouve la classe **Joueur**, qui hérite de la classe **Personnage**, tout

comme les `Ennemi` et les `PNJ`. Cela permet de partager des comportements communs comme le déplacement, la gravité ou la gestion de la vie. Les `PNJ` sont eux-mêmes divisés en plusieurs types comme `PnjJake`, `PnjDonut`, ou encore `DentifriceVolant` et `BrosseADent`. À côté de ça, on retrouve tous les objets utilisables dans le jeu (pioche, épée, arc...), qui héritent tous de la classe abstraite `ObjetUtilisable`. Ces objets sont stockés dans l'`Inventaire`, qui est directement lié au `Joueur`. Le `Terrain` est aussi une pièce centrale : il contient les `Ennemi`, les `Projectiles`, et il est modifié par le `Terraformer`, par exemple quand un bloc est cassé. Les projectiles comme les flèches ou les boules de dentifrice héritent tous de la classe `Projectile`, ce qui permet de gérer leur mouvement, leur portée et les collisions. Le système de craft est géré par le `GestionnaireDeRecettes` et les `Recette`, avec des types de matériaux définis dans l'enum `TypeMateriaux`. Enfin, la gestion de la vie du joueur à l'écran est faite par la classe `VieVue`.

2 ; Vous choisirez des parties du modèle que vous considérez particulièrement intéressantes du point de vue de la programmation à objets (héritage, composition, polymorphisme...).

Vous expliquerez vos choix et les illustrerez par des diagrammes de classe.

Chaque diagramme utilisé doit être focalisé sur un objectif de communication. Il ne doit par exemple pas forcément montrer toutes les méthodes et dépendances, mais juste ce qui est nécessaire pour montrer ce que le diagramme veut montrer. Chaque diagramme doit être commenté



On a une classe principale qu'on appelle Personnage. C'est une sorte de modèle de base que tous les personnages du jeu suivent. Cette classe contient les éléments communs à tous, la position sur l'écran (x, y), la vie, le fait de tomber à cause de la gravité, ou encore se déplacer. Grâce à ça, on ne répète pas le même code partout. Ensuite, on a des classes qui héritent de Personnage. Par exemple, la classe Joueur hérite de Personnage, et ajoute le fait sauter ou utiliser des objets. De l'autre côté, on a la classe Ennemi, le DentifriceVolant et la BrosseADent peuvent attaquer le joueur quand il est proche. On a aussi une classe spéciale Pnj, qui est encore une classe de base, mais pour les personnages non jouables. Dedans, on a mis des PNJ différents, comme Pnj Jake qui lorsque le joueur s'approche il effectue une action, ou Pnj Donut qui se déplace plus simplement. Tout ça, ça montre bien l'héritage, parce que chaque personnage hérite des choses de celui d'au-dessus. Ce qui veut dire qu'on peut appeler une fonction comme seDeplacer() sur n'importe quel personnage, même si on ne sait pas exactement lequel c'est. Ça nous permet de gagner du temps, d'écrire moins de code, et surtout de pouvoir ajouter facilement d'autres personnages plus tard.

Tests Junit : (15 points pour SAEs S2.01 S2.02)

Une ou deux classes bien choisies devront être accompagnées de tests Junit cohérents et raisonnablement complets.

Les tests junits se trouvent dans le package modèle et se nomme "InventaireTest" et "RecetteTest"

3 Structures de données (15 points pour SAEs S2.01 S2.02)

List<Projectile> dans Terrain et Controleur, pour suivre les projectiles actifs dans le jeu.

List<Ennemi> dans Terrain, pour stocker les ennemis présents sur la carte.

List<Integer> dans Inventaire, pour gérer les indices ou les quantités d'objets.

List<Recette> dans GestionnaireDeRecettes, pour stocker toutes les recettes de craft.

ComboBox<TypeMateriaux> dans CraftingVue, qui repose aussi sur une collection interne pour lister les matériaux.

Iterator dans Terrain qui permet d'avoir accès à l'élément suivant.

4 algorithmique (30 points pour SAEs S2.01 S2.02)

BrosseADent – méthode seDeplacer()

Utilise un BFS (parcours en largeur) pour trouver un chemin vers le joueur en évitant les obstacles du terrain.

Terrain – méthode collision(int x, int y)

Vérifie si une case est vide ou non. Utilisée pour gérer les déplacements et les interactions avec les blocs.

GestionnaireDeRecettes – méthode getRecettePour(TypeMateriaux)

Parcourt la liste des recettes et retourne celle qui correspond au type d'objet voulu. Utilisé pendant le craft.

5 Document utilisateur (40 points pour SAE S2.05) :

Objectif et univers du jeu

.Présentation du jeu avec captures d'écran.

Mécanique des actions utilisateurs.

Caractéristiques des armes et accessoires de Link, des types d'éléments de la map, des ennemis (tableau de relation ...).

Toutes fonctionnalité définie et utilisable doit être documentée. A l'inverse il est hors de question (et donc pénalisant) de documenter une fonctionnalité non encore utilisable

Le fichier est présent dans le zip. "Document utilisateur.pdf"